CoT Commitment Points: Project Development Journal

Project: Mechanistic interpretability of chain-of-thought commitment in Qwen2.5-14B-Instruct **Supervisor:** Prof. D. Pham, University of Birmingham **Author:** Aditya Singh **Document date:** June 2026 **Document type:** Development journal — not the paper, not a results summary. A record of how the project actually moved from its original design to its current state, including everything that went wrong and what each failure revealed.

What This Document Is and Is Not

This is a process record. It exists so that a reader who was not present for any of the debugging sessions can understand why the experiment currently running is not the experiment that was originally proposed, why the paper's early sections (specifically Section 5, the pilot table) no longer describe the actual methodology, and why the project's central claim has changed shape twice.

It is not a substitute for the paper. The paper will report a final result with controls and confidence intervals. This document reports the *path* to that result, including dead ends, and is written at the level of specificity a supervisor needs to evaluate whether the path was sound — not the level of polish a reviewer expects from a submission.

Two items below are marked `[VERIFY]`. They are claims that appear plausible and consistent with the project's trajectory but have not been confirmed against the actual code history available at the time of writing. They are included because removing them would silently erase part of the record, but they should not be repeated in the paper or to Prof. Pham without confirmation.

Section 1 — The Original Design

The question

The project's question came from a gap between complexity theory and empirical behavior. Merrill and Sabharwal (2023, arXiv:2310.07923) established that a transformer's single forward pass is bounded by TC^0, and that chain-of-thought — a linear number of intermediate generation steps — is what lifts the effective computational class beyond that bound. This is a result about *capacity*: CoT makes more computation possible. It says nothing about *where in the trace* that computation happens, or whether a trace that has gone wrong can be corrected once it is underway.

The project's question filled that gap: at what semantic stage of a chain-of-thought trace does an incorrect trajectory become causally unrecoverable, even under direct intervention

on the model's internal representations? "Unrecoverable" was operationalized as: replacing the residual stream activations at that stage with activations from a correct trace on the same problem fails to redirect the model to the correct final answer.

The original stack

- **Model:** Qwen2.5-1.5B. Small enough to run in float16 without quantization, large enough to produce genuine multi-step reasoning.
- **Library:** TransformerLens, chosen specifically because it is built for mechanistic interpretability — hook-based activation access with a clean API designed for exactly this kind of intervention.
- **Dataset:** GSM8K, restricted to the hard tail — test problems 900 through 1319, the 419 problems with the highest greedy error rate.
- **Method:** Classic activation patching. Run the wrong trace through the model with teacher forcing (the full trace as input, no generation). Separately store the residual stream activations from a correct trace on the same problem. Inject the correct trace's activations at the stage of interest, across all layers, into the wrong trace's forward pass. Read the logit-difference shift (Δld) at the answer-token position — the change in log P(correct) − log P(wrong) caused by the patch.
- **Stages:** Four, defined independently — setup, computation, transition, conclusion. Each patched separately; results compared across the four.
- **Target N:** 160 pairs (80 correct, 80 incorrect).
- **Deliverable:** A commitment curve — Δld as a function of stage — predicted to be monotonically declining. High recoverability at setup (nothing has been decided yet). Peak recoverability at computation (this is where arithmetic errors form, and correcting them should matter most). A drop at transition (the "therefore" tokens are announcing a conclusion already reached internally). Near zero at conclusion (the answer is already written).

The N=2 pilot

Two pairs were run under this design. The result:

Stage	Δld shift
Setup	+14.6
Computation	+13.9
Transition	+0.3

This looked like exact confirmation of the predicted curve. Setup and computation both showed large, comparable recoverability; transition showed almost none. The interpretation written at the time — "by the time the 'therefore' tokens appear, the trajectory has already committed; the verification-friendly checkpoint is structurally too late" — became the project's headline claim. It went into the cold-outreach research statement (the Maddison email), into the slides prepared for Prof. Pham, and into Section 5 of the paper draft.

As of this writing, Section 5 of the paper PDF still contains these three numbers. Every subsequent section of this document explains why they cannot remain there.

Section 2 — The Four Pivots

Four things changed between the original design and the experiment currently running. None of them were planned in advance; each was forced by something the previous design couldn't handle. They compound — understanding the current notebook requires understanding all four, in order.

Pivot 1 — Model: 1.5B → 14B-Instruct, NF4

`[VERIFY]` The stated reason for this pivot is that TransformerLens does not support NF4 quantization, and that loading Qwen2.5-1.5B in float16 left no VRAM headroom on a T4 for activation storage during patching. This is consistent with everything downstream — the project does in fact run on raw PyTorch forward hooks, not TransformerLens — but the specific causal claim (TransformerLens + NF4 incompatibility as the forcing event) has not been checked against the actual commit history and should be confirmed before it appears in the paper's methods section or is stated to Prof. Pham as the reason for the model change.

What is *not* in question is what resulted. The project now runs entirely on hand-written activation capture and injection:

- `store_pass`: runs the correct trace through the model, captures the residual stream output of every one of the 48 decoder layers at every token position, and stores it.
- `patch_pass` (the hook infrastructure inside `run_experiment` and `run_per_layer`): registers a forward hook on each target layer. The hook receives the layer's output for the wrong trace's forward pass, clones the hidden state tensor, and for each position in the patch window, overwrites that position's hidden state with the corresponding position's stored activation from the correct trace. The closure pattern (`def mk(i): def hook(...): ... return hook`) correctly captures the layer index per-hook — this was checked directly against the notebook and is correct, with bounds checking on both the target and source position indices.

This infrastructure took substantially longer to build than using TransformerLens's existing hook API would have, and several of the bugs catalogued in Section 3 originate directly in this hand-written layer — the `p_len` recomputation issue, the cross-problem position-indexing fallback, and the cumulative-mask clipping bugs are all artifacts of writing activation patching from scratch rather than using a library that handles position bookkeeping internally.

What the pivot bought back: Qwen2.5-14B-Instruct produces qualitatively better-structured reasoning traces than 1.5B. Computation stages are more clearly delineated; setup is more explicit. The traces are more analyzable. This is a real benefit, but it came at the cost of months of infrastructure work that a smaller model with library support would not have required.

Pivot 2 — Dataset: GSM8K → MATH Level 4-5

This pivot happened because Qwen2.5-14B is, for this project's purposes, too good at GSM8K.

Temperature sampling at T=1.4 on the GSM8K hard tail still produced correct answers more than 80% of the time. Pushing temperature higher to force more wrong traces produced incoherent text — not genuine reasoning errors, just noise. Worse, the greedy failures that did occur on the hard tail were what the project later termed "soft failures": the model's reasoning was correct throughout, and it simply emitted the wrong final token. A teacher-forced forward pass on these traces already assigned higher probability to the correct answer *before any patching occurred* — there was nothing for the patch to fix. Of the first 8 pairs collected under the original design, 7 failed this pre-check.

MATH Level 4-5 (DigitalLearningGmbH/MATH-lighteval) has a genuine greedy error rate around 60% — high enough that wrong traces collected from it represent the model actually going down a structurally incorrect reasoning path, not clipping a final digit.

The cost of this switch was textual: the paper's dataset description, every example, every number in the existing PDF refers to GSM8K. The switch to MATH was not propagated into the paper until very late, and as of this writing every instance of "GSM8K" in the paper needs to become "MATH Level 4-5."

The switch also revealed three structural properties of MATH that the segmentation classifier was not designed for:

1. MATH answers are wrapped in `\boxed{...}`, often with nested braces (`\boxed{\frac{3}{4}}`), which GSM8K's `#### number` format does not have.
2. MATH answers are frequently non-integer — fractions, expressions — which broke assumptions in the answer-normalization code that had been written against GSM8K's integer-only answers.

3. MATH solutions frequently *open* with transition-patterned phrasing — "To solve this," "Let us first determine" — which the regex classifier built for GSM8K's structure tends to label as `transition` rather than `setup`. This causes the `setup` token mask to collapse toward empty for a meaningful fraction of MATH pairs, which is directly relevant to why `setup`'s patched-pair count (`n=4`, see Section 4) is so small.

Pivot 3 — Method: Teacher-forced logit inspection → Truncate & Generate

This is the most consequential pivot, and the one with the deepest implications for what the experiment can claim.

The teacher-forcing problem. In the original design, the full wrong trace is given to the model as input, and the logit at the answer position is read. But the model's attention at that position can see every token before it — including all of the wrong reasoning. Patching the residual stream at, say, the computation-stage positions while every token *after* those positions still contains the original wrong computation text creates an incoherent signal: you are telling layer-N's representation at position P "here is what a correct computation looks like," while the attention mechanism at every later position is simultaneously reading the actual (wrong) computation text that followed position P in the original sequence. The patch and the visible context disagree with each other, and it is unclear what such a patch is actually testing.

The already-flipped problem, mentioned above, is the same issue from a different angle: teacher-forcing on a trace whose error is "soft" can show the model already favoring the correct answer internally, even while the trace as written produces the wrong token. 7 of the first 8 collected pairs failed this check under the original design.

Truncate & Generate (T&G) resolves both. The wrong trace is cut at the semantic stage boundary. A clean baseline generation is run from that truncation point — no patching — and the resulting answer is checked. If the model self-corrects on its own (produces the correct answer with no intervention), the pair is excluded as `already_flipped`: there is nothing for a patch to demonstrate, because the model didn't need help. If the model does *not* self-correct — it commits to the wrong answer even from a clean truncation point — then the patched generation is run, with correct-trace activations injected into the prefix via the same hook infrastructure, and the result is checked for a flip.

The already-flipped check, which was a fragile pre-screening step in the original design, becomes a natural and informative consequence of the T&G methodology: it directly measures how many wrong traces are recoverable *without any activation intervention at all*, just by truncation and regeneration. This number turns out to be large — see Section 4 — and is itself one of the project's findings.

What this pivot costs conceptually: logit inspection asks "does this patch shift the internal probability distribution at a fixed position?" T&G asks "given a corrected prefix and a fresh start, can the model reach the correct answer?" These are related but not the same question.

The paper's Section 4.5, as of this writing, still describes logit inspection. The code runs T&G. This discrepancy existed in every submitted notebook version until the methodology section is rewritten — which has not yet happened.

Pivot 4 — Stage structure: four independent stages → five cumulative stages

The original design patched setup, computation, transition, and conclusion independently and compared the four results. Under T&G, independent patching creates a new problem: if "computation" is patched in isolation but the truncation point is computed independently for each stage, the truncation point for "computation alone" may fall in an incoherent place — mid-sentence, or before the model has even reached the part of the trace that establishes the problem.

The fix was a cumulative design: `pre_setup` (the first ~10% of the trace), `setup`, `computation_only` (computation tokens specifically, excluding setup), `+computation` (setup and computation together), `+transition` (setup, computation, and transition together). Each cumulative stage gives the model a coherent prefix — everything up to and including that point in the trace — to generate forward from.

`computation_only` was then added as a sixth condition, separate from the cumulative ladder, for a specific reason discovered during analysis: in MATH traces, setup tokens are not contiguous — they scatter throughout the trace (a symptom of the transition-phrased-opening issue from Pivot 2). This means the *last* setup-labeled token can occur *after* the last computation-labeled token. When that happens, `+computation`'s truncation point — defined as extending through all setup and computation tokens — lands later in the trace than `computation_only`'s truncation point, potentially *after* the wrong `\boxed{...}` answer has already been written into the visible prefix. A patch cannot un-write a token that's already in the context. `computation_only`, which truncates based on computation tokens specifically, does not have this problem to the same degree.

The net effect of Pivot 4: the results table has five labeled rows, but they collapse to two independent experimental conditions. `computation_only` is the clean isolated condition. `+computation` and `+transition` are, empirically, the same condition — a Tier-1 check confirmed the identical set of pairs flips at both, meaning transition tokens add nothing once setup+computation is already in the patch window. `pre_setup` and `setup` are degenerate in a different way — see Section 4.

Section 3 — The Bug Timeline

This section is a chronological log of every confirmed bug found during the project, in roughly the order each was found. Each entry states the location, the symptom, the fix, and — critically — which prior results the bug invalidates.

dtype vs torch_dtype. `AutoModelForCausalLM.from_pretrained()` was called with `dtype=torch.float16`, which is not a valid parameter for that function and is silently ignored (generating a deprecation warning in current `transformers`). Fix: `torch_dtype=torch.float16`. Impact: low on its own, because NF4 quantization sets its own compute dtype separately — but see the next entry.

`[VERIFY — Bug "0"]` `bnb_4bit_compute_torch_dtype`. The corrected parameter name from the previous fix was reportedly mirrored incorrectly into `BitsAndBytesConfig`, producing `bnb_4bit_compute_torch_dtype` — a field that does not exist in that config class, which would silently fall back to float32 compute. Float32 activation storage during `store_pass` uses roughly double the VRAM of float16, which on a 16GB T4 running a 14B NF4 model could plausibly cause an OOM mid-experiment, or simply double the memory footprint of every stored activation tensor without crashing. **This specific bug name has not been confirmed against the actual notebook history available at the time of writing.** What *is* confirmed: the current notebook (`notebook_publishable.ipynb`) correctly sets `bnb_4bit_compute_dtype=torch.float16`. Whether an intermediate version had this exact typo, or whether this entry conflates it with the `dtype`/`torch_dtype` issue above, should be checked before this appears in the paper.

Segmentation drift from `re.split`. `get_stage_masks` originally split the wrong trace into lines using `re.split('\n+', wrong_trace)`. Consecutive newlines in the trace produce empty strings in the split result, which shifts the running character-position counter used to map each line back to its token indices — every line after the first blank-line region gets attributed to the wrong character range, and therefore the wrong token range, and therefore the wrong stage. Fix: replace the split with a character-exact walk using `wrong_trace.find('\n', cur_in_trace)`, which never loses position. Impact: every stage-label result produced before this fix has potentially incorrect token-to-stage attribution. This is a foundational bug — it affects every downstream stage-conditioned measurement from versions before the fix.

Setup truncation: max vs first-non-setup. `build_cumulative_masks` originally computed the setup truncation point as `max(setup_token_indices) + 1` — i.e., one past the *last* token labeled setup. Because setup tokens scatter (Pivot 2/4), this could place the truncation point very late in the trace — effectively including large stretches of non-setup material in what was supposed to be a setup-only truncation. Fix: the truncation point is the index of the *first* non-setup token, `non_setup[0]`. Impact: prior to this fix, `setup`-stage truncation was not testing what it claimed to test.

Cumulative masks not clipped to source-trace length. When the correct trace (the source of injected activations) is shorter than the wrong trace (the target being patched), a patch position computed from the wrong trace's length could index beyond the end of the correct trace's stored activations — an out-of-bounds read. Fix: clip cumulative masks to `c_trace_len` (the correct trace's token length) before computing truncation points.

Shuffled-positions control not clipped. The same out-of-bounds issue, specific to the shuffled control: random positions were drawn from the *wrong* trace's full length, but the source activations being injected only extend to `c_trace_len`. Fix: clip the valid sampling range to `min(trunc_len - p_len, c_trace_len)`.

`normalize_answer` **vs** `_clean_num`**.** Two competing answer-normalization functions coexisted in the codebase with different edge-case handling for fractions, LaTeX, and percentages — meaning which function ran for a given comparison depended on call-site, not on any principled choice. Fix: remove `_clean_num` entirely; `normalize_answer` is now the single canonical normalizer. Impact: some pairs in earlier runs may have been classified as flipped or not-flipped depending on which normalizer happened to be invoked for that comparison.

`p_len` **recomputed inside the patching loop (**`run_per_layer` **only).** `p_len` — the prompt length in tokens — was computed once per pair at the start, then *recomputed* inside the hook function. For the same-problem case this produces an identical value and is harmless. For the cross-problem case, the prompt differs (a different question), so the recomputed `c_p_len` inside the hook diverges from the pre-computed `p_len` used elsewhere. `run_experiment` handled this correctly; `run_per_layer` did not. Fix: use the pre-computed values consistently throughout `run_per_layer`.

`\boxed{}` **parsing with nested braces.** `extract_gt` originally used `r'\boxed{([^}]+)}'`. MATH answers routinely nest braces — `\boxed{\frac{3}{4}}` — and `[^}]+` stops at the *first* closing brace, capturing `\frac{3` instead of `\frac{3}{4}`. Fix: character-by-character brace-depth counting until the matching close brace is found.

The critical bug — `max_new_tokens=150`**.** Found by an external audit (Gemini), after multiple internal analysis passes had missed it. All four `model.generate()` calls across `run_experiment` and `run_per_layer` were hardcoded to `max_new_tokens=150`. Truncating at `pre_setup` or `setup` and asking the model to complete a MATH Level 4-5 solution typically requires 500–1500 tokens. At 150 tokens, generation is cut mid-solution, `extract_gt` finds no `\boxed{}` or `####` and returns `None`, and the surrounding code interprets `None` as `already_flipped` or `skip` — silently. This means every `n`, every `already_flipped` count, and every flip rate from before this fix conflates two different things that look identical in the output: pairs that genuinely self-corrected, and pairs whose generation was simply cut off before an answer could appear. Impact: **every result from every run before this fix is not comparable to results after it.** The pre-fix `already_flipped=4` at both `pre_setup` and `setup` became `14` and `15` respectively after the fix (Section 4, State 2 → State 3) — a roughly 3.5–4× change, consistent with the hypothesis that most of the pre-fix exclusions were token-limit artifacts, not genuine self-corrections. Fix: replace `150` with `MAX_NEW_TOKENS = 2048` in all four call sites; confirmed present and in-scope in the current notebook.

Regex backslash count for `\boxed{}` classification — proposed, not yet confirmed. To prevent `\boxed{wrong_answer}` from being misclassified into `setup` or `computation` (which would put the literal wrong answer into a patched prefix, making recovery structurally impossible regardless of patch content), a new pattern was proposed for the conclusion-stage `PATTERNS` entry: `r'\\\\boxed\{'`. As written, this is **four** backslashes in source, which as a regex matches the literal text `\\boxed{` — a double backslash. `tokenizer.decode()` output contains a *single* backslash before `boxed`. The pattern that matches a single literal backslash is `r'\\boxed\{'` — two backslashes in source. The four-backslash version compiles without error, runs without error, and matches nothing — leaving the original misclassification problem exactly as it was, invisibly. **As of this writing, this has not been tested.** The correct choice depends on how the existing `PATTERNS` entries for other LaTeX commands (`\frac`, `\sqrt`, etc.) are written — whichever convention they use, `\boxed` should match it. A ten-second check (`re.search(pattern, sample_pair_text)` for both candidate patterns against a pair known to contain `\boxed{}`) resolves this before any further run.

Cross-problem control: adjacent-pair selection. The cross-problem control originally selected the "different problem" as `(pi + 1) % len(pairs)` — the immediately adjacent pair in the collected list. Pairs collected sequentially from a filtered dataset are more likely to be similar to their neighbors than to a random pair, which weakens the control's claim to be testing a "maximally different" problem. Fix: `(pi + len(pairs) // 2) % len(pairs)` — selects a pair from roughly the opposite end of the collection.

Top-up collection without deduplication. When `pairs.json` already contains some pairs and more are needed to reach a target N, the top-up collection call originally restarted from the beginning of the dataset with no memory of which problems were already used — risking duplicate pairs. Fix: build a set of already-used question identifiers before top-up collection and filter against it.

`ds_hard` undefined on top-up path. If `pairs.json` exists, the notebook loads it and skips the cell that defines `ds_hard` (the filtered dataset). If the top-up condition then triggers, `ds_hard` is referenced before assignment — a `NameError`. Fix: an explicit guard inside the top-up block reloads the dataset if it isn't already in scope.

Regex test cell may produce a false negative. A diagnostic cell tests the `\boxed{}` regex against `pairs[0]`. If pair 0 happens to use `#### number` format rather than `\boxed{}`, the test returns `None` for *both* the broken and fixed patterns — and a developer could spend time debugging a non-issue. Fix: search across all pairs for one containing `boxed` before running the diagnostic.

Segmentation validation cell can pass silently with zero output. The validation loop only prints when a line contains `boxed` or `####`. If the first few pairs happen to use only `####`, the loop produces no output at all for the `\boxed{}` case — and zero output is visually

indistinguishable from "checked, and everything passed." Fix: assert that at least one `\boxed`-containing line was actually checked.

`greedy_budget` too tight for top-up. Observed collection ratio during the main run was roughly 30 problems scanned per usable pair collected. Topping up by 21 pairs therefore requires scanning roughly 630 problems; the top-up `greedy_budget` was set to 300 — less than half of what's needed. Fix: raise to 500 for top-up collection.

Timing-estimate constant in `run_per_layer` was wrong by 3–5×. The estimate `len(subset) * len(layers_to_run) * 8 / 60` assumes 8 seconds per `generate()` call — a number calibrated against GSM8K. On MATH Level 4-5 with Qwen2.5-14B NF4 on a T4, observed per-call time is 20–45 seconds. A run estimated at 12 minutes under the old constant runs closer to 90–110 minutes in practice. Fix: change the constant to 22 (a rough midpoint). Impact: multiple sessions hit Kaggle's time limit mid-run because the displayed estimate was 3–5× too optimistic.

Inverted flip metric for the control experiment (found in the same external audit as `max_new_tokens`). `run_experiment` determined "flip" using `answers_match(patched_pred, gt)` for both the main experiment and the control. For the main experiment (correct activations patched into a wrong trace), this is correct — a flip means the output now matches ground truth. For the control (wrong activations patched into a *correct* trace), the baseline is already ground truth; if the model resists the patch and still outputs ground truth, the old metric counted this as a "flip" — when it is in fact the *good* outcome (robustness). A perfectly robust control would have shown 100% "flip rate" under the old metric, when it should show 0%. Fix: for the control, a flip is `not answers_match(patched_pred, gt)` — i.e., corruption away from ground truth. Why prior results were coincidentally unaffected: all 10 control pairs in the relevant earlier run produced ground-truth output regardless, so both the old (inverted) and new (correct) metric produced the same number on that specific data — the bug was real but had not yet been exposed by a control pair that actually got corrupted.

Section 4 — Results Evolution

This section tracks every distinct numerical state the project has been in, in order, with the reason each transition happened.

State 1 — N=2, logit inspection, GSM8K

Δld: +14.6 (setup), +13.9 (computation), +0.3 (transition). Already addressed in Section 1. N=2 is not a sample size from which any conclusion can be drawn; the methodology (logit inspection) is no longer the methodology in use; the dataset (GSM8K) is no longer the dataset in use. These three numbers cannot be cited anywhere going forward, including in the paper's Section 5 where they currently still appear.

State 2 — N=19, Truncate & Generate, 150-token cap, uncorrected BNB dtype

Stage	Flip rate	n	already_flipped
pre_setup	0%	—	4
setup	0%	—	4
computation_only	57.1%	14	—
+computation	35.3%	17	2
+transition	35.3%	17	2

The headline from this state was the apparent non-monotonicity: `computation_only` (57.1%) exceeded `+computation` (35.3%), interpreted at the time as "adding setup tokens to the patch actively hurts recovery." This interpretation does not survive State 3 — the gap closes entirely once the 150-token cap is fixed, which is consistent with the cap differentially truncating `+computation`'s longer required generations more severely than `computation_only`'s shorter ones. The non-monotonicity was very likely an artifact of the bug, not a finding about the model.

The cross-problem control (0% at `+computation`) and shuffled control (47.1% at `+computation`, exceeding the structured 35.3%) from this state are both invalid for the same reason — both were run under the 150-token cap.

State 3 — N=19, Truncate & Generate, 2048-token cap, corrected BNB dtype (current `notebook_publishable.ipynb` Block A)

Stage	total	flips	skip	already_flipped	Flip rate
pre_setup	5	3	0	14	60.0%
setup	4	0	0	15	0.0%
computation_only	12	6	1	6	50.0%
+computation	14	7	0	5	50.0%
+transition	14	6	0	5	42.9%

Three things changed from State 2:

First, the `already_flipped` counts at `pre_setup` and `setup` roughly quadrupled — from 4 to 14 and 15 respectively. With the full 2048-token budget, 14 to 15 of the 19 collected wrong traces *self-correct with no patching at all* when truncated at the very beginning of the trace and given a fresh generation. This is itself a result — see Section 6.

Second, the non-monotonicity from State 2 disappeared. `computation_only` (50.0%) and `+computation` (50.0%) are now equal. The State-2 gap was the artifact described above.

Third, `+computation` (7 flips) and `+transition` (6 flips) now differ by exactly one pair, out of the same `total=14`. One pair flips under `+computation`'s patch but not under `+transition`'s. At n=14 this is most plausibly a single noisy event, but it should be checked before any claim that the two stages are "identical."

State 4 — Per-layer scan, `computation_only`, 13 sampled layers, n=12 (Block C)

Layer	0	4	8	12	16	20	24	28	32	36	40	44	47
Flip rate	33%	33%	25%	42%	33%	33%	42%	25%	25%	8%	8%	0%	0%

This is the first scan with any shape to it. Layers 0 through 32 sit in a noisy band roughly 25–42% with no single standout layer — not a localized "hump." Layers 36 and above drop sharply: 8%, 8%, 0%, 0%. The drop from layer 32 (25%) to layer 36 (8%) to layer 44 (0%) is the single largest effect-size change anywhere in the dataset to date.

Two confounds apply to this curve and neither has been resolved:

- **Propagation depth.** A single-layer patch at layer L affects everything from L+1 through 47 within that forward pass, via the residual stream. A patch at layer 0 has 47 subsequent layers to integrate the correction; a patch at layer 47 has none. A monotonic decline toward later layers is the *expected shape* of this design even if no layer is mechanistically special — early layers simply have more downstream computation available to make use of the correction.
- **The Block D floor (below).** Whatever "no real effect" looks like for this design has not been independently established at the same n as this scan.

Block D — correct-group, NF4 artifact control, n=5, same 13 layers, wrong-trace activations injected into correct traces

Layer	0	4	8	12	16	20	24	28	32	36	40	44	47
Corruption rate	20%	60%	40%	40%	60%	40%	20%	40%	60%	20%	20%	20%	20%

This control was designed to establish a near-zero floor — if single-layer patches don't trivially destabilize a correct trace, the State-4 curve can be read as signal. Instead, the floor ranges from 20% to 60%, with a *minimum* of 20% across all 13 layers. State 4's signal ranges 0–42%. The two ranges overlap substantially. At n=5, each value moves in 20-percentage-point steps, so this result cannot be quoted precisely — but its *direction* matters regardless of precision: it does not establish the near-zero floor it was designed to establish, and until it does, no point on the State-4 curve can be confidently called "signal" rather than "floor."

State 5 — Cross-problem control, same 19 pairs (Block B)

Stage	Within-problem (Block A)	Cross-problem (Block B)
computation_only	50.0% (6/12)	21.4% (3/14)
+computation	50.0% (7/14)	**0.0% (0/17)**
+transition	42.9% (6/14)	**0.0% (0/17)**

The `+computation`/`+transition` contrast — 50% and 42.9% within-problem versus 0% cross-problem — is the largest and cleanest gap in the dataset. It strongly supports the claim that the recovery effect requires semantically coherent, problem-specific activation content, not just any large injected signal.

The open issue State 5 introduces: baseline reproducibility

The `already_flipped` count for the *same stage, same 19 pairs, same no-patch baseline check* differs between Block A and Block B:

Stage	already_flipped (Block A)	already_flipped (Block B)	Difference
pre_setup	14	14	0
setup	15	15	0
computation_only	6	4	2
+computation	5	2	3
+transition	5	2	3

The `already_flipped` check does not depend on whether the subsequent patch is within-problem or cross-problem — it is a no-patch generation from the truncation point, full stop. It should be identical between Block A and Block B. At `pre_setup` and `setup` it is. At

`computation_only` and the cumulative stages it is not — by 2 and 3 pairs respectively, out of 19.

The practical consequence: the `+computation` cross-problem result (0/17) and the `+computation` within-problem result (7/14) are computed over denominators that differ by 3 pairs — and those 3 pairs are in the denominator *because* Block B's baseline check classified them differently than Block A's did, for reasons that have nothing to do with cross-problem vs. within-problem patching. The headline 50pp gap at `+computation` is real in direction, but its exact magnitude rests on a baseline check that is not reproducible run-to-run. The most likely causes are non-deterministic generation (an unconfirmed `do_sample=True` somewhere) or fp16 accumulation drift over a 2048-token generation occasionally flipping an argmax decision at some branch point. Neither has been checked.

This issue does not appear in any prior synthesis of this project's results, including the most thorough one produced so far. It was found by comparing Block A's and Block B's raw `already_flipped` counts side by side — a check that does not appear if one only compares the two blocks' headline flip-rate percentages, which is how every previous synthesis compared them. See Section 9.

Section 5 — The Controls Story

The N=2 pilot had zero controls. The first formal review identified this as the project's central weakness: without a control, "the patching worked" cannot be distinguished from "truncation alone produces this recovery rate" or "any sufficiently large disruption to the forward pass redirects the model to the correct answer about a third of the time, regardless of content."

Three controls were added, in this order:

Correct-group retention. Patch correct-trace activations into an already-correct trace. Expected result: the trace remains correct — the hook mechanism does not corrupt correct reasoning. Result: 100% retention across all stages. This validated the basic machinery — but, as noted in Section 3, the metric used to compute this number was *inverted* at the time, and the 100% result was coincidentally correct because all 10 control pairs happened to produce ground-truth output regardless of which direction the metric counted. The control passed; the reason it appeared to pass was not yet the right reason.

Cross-problem null. Patch activations from a *different problem's* correct trace into the wrong trace. Expected result: near 0% — rules out "any large injection works regardless of content." Result (State 5, current run): 21.4% at `computation_only`, 0% at `+computation` and `+transition`. The within-problem effect is real and at least partially problem-specific.

Shuffled positions. Patch the *same* problem's correct-trace activations, but at randomly chosen positions rather than the semantically-matched ones. Expected result: lower than the structured (position-matched) result — distinguishes "which positions matter" from "how much is patched." Under the State-2 (pre-fix) run, this produced the anomalous result of shuffled (47.1%) *exceeding* structured (35.3%) at `+computation` — which, like the State-2 non-monotonicity itself, is most likely an artifact of the 150-token cap differentially affecting the two conditions' effective truncation points. This control has not yet been re-run under the current (fixed) notebook.

Together these form a 2×2 design — content specificity (same problem vs. different problem) crossed with position specificity (structured vs. shuffled positions). As of this writing, only `computation_only` has clean evidence on the content-specificity axis (21.4% cross-problem vs. 50% within). The position-specificity axis has not been re-tested since the 150-token fix, and the Block D floor issue (Section 4) means even the content-specificity result at `computation_only` cannot yet be cleanly separated from "any single-layer-scale disruption produces some baseline rate of output change."

Section 6 — How the Research Question Changed

What "commitment" meant originally

A single point in the trace, past which the full-residual-stream patch fails to redirect the answer. The commitment curve was expected to be monotonic: high recoverability early, collapsing to near-zero by the transition stage.

What the data says instead

There appear to be two separate phenomena, previously collapsed into one concept.

Context suppression. When a wrong trace is truncated very early (`pre_setup`, `setup`) and the model is simply given 2048 tokens to generate forward — *with no activation patching at all* — 14 to 15 of 19 pairs (roughly 74–79%) produce the correct answer on their own. The model's own wrong reasoning, when it is no longer present in the visible context, stops suppressing the model's ability to reach the correct answer. Nothing about the residual stream needed to change; removing the wrong *text* was sufficient.

Residual-state commitment. The remaining 26–32% of pairs do *not* self-correct even when truncated early and given a fresh generation budget — something about their state at that point in the trace still points toward the wrong answer, independent of the visible text. Of these residually-committed pairs, patching `computation_only` activations recovers roughly half (50%, State 3).

The original framing treated "commitment" as one thing, measured by one curve. The data suggests two different recovery mechanisms with two different practical implications: context suppression is addressed by re-prompting or simply truncating and regenerating — no internals access required. Residual-state commitment requires the kind of activation-level intervention this project is built around, and only addresses it for about half of the pairs where it's present.

The question as it currently stands

Computation-stage residual stream activations in Qwen2.5-14B-Instruct carry recoverable causal information for roughly half of the wrong MATH Level 4-5 traces that do not already self-correct under truncation alone — and the within-problem vs. cross-problem gap at the cumulative stages (50% vs. 0%) suggests this information is at least partially problem-specific, though the magnitude of that gap is not yet on solid footing (Section 4's open issue). Whether this signal is concentrated at particular layers, or broadly redundant across layers 0–32 with a falloff after 36, is the question Block C's curve was meant to answer and has answered only provisionally, pending the Block D floor resolution.

Section 7 — Literature Connections

Merrill and Sabharwal (2023, arXiv:2310.07923). Establishes that a transformer's single forward pass is bounded by TC^0, and that a linear number of CoT steps is what extends this bound. This is the project's theoretical motivation — it establishes that the computation genuinely happens somewhere in the trace, which makes "where" a well-posed empirical question. It is motivation, not a result the experiment tests or confirms directly; the paper should not claim to "validate" this theory, only to be motivated by it.

Turpin, Michael, Perez, and Bowman (2023, arXiv:2305.04388, NeurIPS 2023). Demonstrates that chain-of-thought explanations can systematically misrepresent the actual basis for a model's prediction — when models are biased toward a wrong answer by reordering few-shot examples, they generate CoT explanations supporting that wrong answer without ever mentioning the feature that actually influenced them, and accuracy on BIG-Bench Hard tasks drops by as much as 36%. This is the foundational justification for looking at *activations* rather than trusting the *text* of the CoT — if the text can misrepresent the mechanism, the mechanism has to be examined directly.

Lanham et al. (2023, arXiv:2307.13702). Measures CoT faithfulness by directly intervening on the CoT text — adding errors, paraphrasing — and observing how much the final answer depends on the (possibly corrupted) stated reasoning. Finds large task-dependent variation, and that larger, more capable models tend to produce *less* faithful reasoning on most tasks tested. Reinforces Turpin's point from a different angle and is part of the project's justification for an activation-level rather than text-level approach.

Bogdan, Macar, Nanda, and Conmy (2025, "Thought Anchors," arXiv:2506.19143). A black-box, sentence-level method: resample replacement sentences for a given step in a CoT trace, filter for ones that are semantically different, continue the trace from there, and measure how much the final-answer distribution shifts — a counterfactual measure of that sentence's causal importance. Their broader argument is that reasoning traces should be analyzed at the sentence level the way circuits are analyzed at the component level in standard interpretability.

This paper has a documented history of being misread in this project's own draft. **The paper's Section 11, as of this writing, states that Bogdan et al. show forcing a model to say "the answer is wrong" doesn't change its final answer, and that this implies CoT is causally inert.** That is not what the paper argues or finds — their entire contribution is a method for identifying *which* sentences *are* causally decisive, not a claim that none are. This needs to be corrected before Section 11 is shown to anyone. The genuinely interesting connection — and a more defensible one than the current Section 11 — is that Bogdan et al. operate at the text/sentence level while this project operates at the residual-stream level, and Turpin's unfaithfulness result is precisely what would let these two levels disagree: a sentence can be causally unimportant *at the text level* while the hidden states computed while producing it are causally decisive. If this project's computation-stage result holds up, it is evidence for exactly that kind of divergence — which would make Bogdan et al. a point of productive tension to engage with directly, not a result to (incorrectly) cite as supporting evidence.

Afzal, Matthes, Chechik, and Ziser (2025, arXiv:2505.24362, ACL 2025). Shows that probing classifiers trained on a model's *initial* representations — before any CoT tokens are generated — predict whether a zero-shot CoT attempt will succeed, at 60–76.4% accuracy. This is read-only (probing) rather than causal (patching), but it points in the same direction as this project's `already_flipped` result: information about whether a trace will ultimately succeed appears to be present in the model's state very early, before the reasoning that ostensibly produces the answer has been written out.

Stolfo, Belinkov, and Sachan (2023, arXiv:2305.15054, EMNLP 2023). The closest direct methodological predecessor. Uses causal mediation analysis — intervening on specific model components and measuring the resulting change in predicted probabilities — to identify which parts of a transformer are responsible for arithmetic predictions, finding that relevant information is transmitted from mid-sequence early layers to the final token via attention. Their granularity is per-component (attention heads, MLP layers) at fixed positions; this project's granularity is per-semantic-stage across all layers (Block A/B) or per-layer at one stage (Block C/D). The per-layer analysis, if it survives the Block D floor issue, would be the first point where this project's results become directly comparable to Stolfo et al.'s component-level findings.

Meng, Bau, Andonian, and Belinkov (2022, "ROME," NeurIPS 2022). The methodological origin of activation patching as a causal tool — corrupt a run, restore a clean internal state, measure how much of the original behavior is restored. ROME's framing also carries an important limitation this project inherits: a successful patch shows that a component *can* influence the output under intervention, not that it *normally* does during unmodified generation. The paper's claims should be scoped accordingly.

Heimersheim and Nanda (2024, "How to use and interpret activation patching," arXiv:2404.15255). `[VERIFY attribution]` A methodological best-practices reference for activation patching — the choice of corruption method, metric, and patching scope all materially affect conclusions, and the paper recommends running multiple complementary metrics and controls rather than relying on one. This project's three-control design (correct-group, cross-problem, shuffled) is directly in this spirit. Note: an earlier draft of this project's literature list attributed a similarly-described "best practices" paper to "Zhang & Nanda 2024" — this may be the same paper with an author misremembered, or a different paper entirely. The arXiv ID above (2404.15255) was verified directly; "Zhang & Nanda 2024" was not, and the two should be reconciled before citing either in the paper.

Wei et al. (2022). Established the empirical chain-of-thought prompting result — that CoT prompting substantially improves performance on multi-step reasoning benchmarks including GSM8K. This is the empirical phenomenon (CoT helps) that the project's theoretical (Merrill & Sabharwal) and mechanistic (this project's own) framings are both trying to explain.

`[VERIFY — citation not checked]` **A "functional rift" prediction, attributed in an earlier draft to Dutta et al. (2024).** The prediction was that commitment would be *localized* to a specific mid-layer band — a "rift" between early and late layers. The State-4 per-layer curve (flat 25–42% across layers 0–32, dropping after 36) does not show a localized mid-band peak; it shows a broad plateau with a late falloff. Whether this is in tension with, consistent with, or simply orthogonal to whatever Dutta et al. (2024) actually argues cannot be assessed until the citation itself is confirmed — both the authors and the specific claim should be checked before this comparison appears in the paper.

Section 8 — The Statistics

Where N=80 came from

N=80 was never derived from a power calculation. It was chosen as a round number that fit within a single long Kaggle session, under timing estimates that were later found to be wrong by 3–5× (Section 3). It then propagated through every version of the paper as "the" target N without justification.

The exclusion-rate problem

Collected N is not eligible N. At `computation_only` in the current (State 3) run, of 19 collected pairs, 6 are `already_flipped` and 1 is `skip` — 7 of 19, or 37%, are excluded from the flip-rate denominator. The eligible fraction is 63%.

A target of "N=80 collected" therefore yields roughly 50 eligible pairs, not 80. This distinction was not part of the project's planning until late.

A corrected power calculation

For a binomial proportion at p=0.5 (the worst case for confidence-interval width), the 95% CI half-width is `1.96 × sqrt(p(1−p)/n) = 0.98/sqrt(n)`. Solving for n given a target half-width x: `n ≥ (0.98/x)²`.

For a workshop-level target of ±14 percentage points: `n ≥ (0.98/0.14)² = 49`. (An earlier pass at this calculation produced 43; the correct value, recomputed here, is 49 — close, but the correction should be carried forward.) At 63% eligibility, 49 eligible pairs requires collecting roughly 78 — consistent with the "collect ~80" target, though for the right reason now rather than an arbitrary one.

For a main-track target of ±10 percentage points: `n ≥ (0.98/0.10)² ≈ 97`. At 63% eligibility, this requires collecting roughly 154 — consistent with a "collect ~160" target for main-track precision.

Power to detect the within/cross-problem gap

For the 50% vs. 21.4% gap at `+computation`/`computation_only` specifically — a two-proportion comparison rather than a single CI — a standard power calculation for 80% power at α=0.05 gives a required eligible n in the low 40s (roughly 40–42 depending on the exact formula variant used). This is *less* demanding than the ±14pp single-proportion requirement (49), so the single-proportion CI width is the binding constraint, not the gap-detection power.

What N=40 (collected) gives

At 63% eligibility, 40 collected → roughly 25 eligible → 95% CI at p=0.5 of roughly ±19.6pp, i.e., [30%, 70%]. Wide, but tight enough to distinguish "the State-3 result (50%) survives" from "it collapses toward the cross-problem baseline (21%)" — which is the diagnostic question that should be answered before committing to the full N=80–130 collection run.

Section 9 — The Execution Problem

Across this project's history, the ratio of analysis to execution has been roughly 20:1. The pattern, repeated multiple times: run something, receive an analysis identifying 8–12 issues,

fix all of them, receive an analysis of the fixes identifying 3–4 new issues, fix those, run again — or, in several cases, *not* run again, and instead receive another analysis of the plan to run again.

The clearest illustration is the `max_new_tokens=150` bug. It silently invalidated every quantitative result in the project for an unknown stretch of its history. Multiple internal analysis passes — focused on experimental design, controls, the paper's description of the methodology, the BNB dtype configuration — all real issues — never caught it. An external audit found it in a single pass. The internal analyses had a structural bias toward design-level problems (is the experiment asking the right question, are the controls adequate) and away from implementation-level bugs (does this specific line of code do what the surrounding code assumes it does).

This document is itself an instance of the pattern in one specific way, and it's worth naming directly: the most thorough synthesis of this project's results produced prior to this document compared Block A and Block B by their headline flip-rate percentages — 50% vs. 21.4%, 50% vs. 0%, 42.9% vs. 0% — and concluded the cross-problem gap was the project's cleanest result. It was not wrong that the gap is large. But comparing headline percentages across two blocks does not surface that the two blocks' `already_flipped` counts — the thing the percentages are *computed from* — disagree by 2–3 pairs out of 19 for the same nominal baseline check (Section 4's open issue). That discrepancy is only visible if you look at the raw `total`/`already_flipped`/`flips` triples side by side, not at the percentages derived from them. A synthesis built from summaries of summaries can be internally consistent and still miss something a synthesis built from raw outputs would not.

The corrected cycle, stated plainly: run something small. Identify the single most consequential issue in the raw output — not the design, the raw output. Fix that one thing. Run again. The two open issues in Section 4 — the baseline reproducibility gap and the Block D floor — are both addressable by a single ~9-minute run (a self-patch identity check: inject a correct trace's own activations into itself and confirm the output doesn't move). That run has been proposed multiple times across this project's history and has not yet been executed.

Section 10 — Final State and What Remains

Current best evidence

Qwen2.5-14B-Instruct, NF4, MATH Level 4-5, 19 pairs collected, all known implementation bugs fixed except the `\boxed{}` regex (proposed, untested). `computation_only` and the cumulative stages (`+computation`/`+transition`, empirically one condition) show 50% flip rate among residually-committed pairs (those that don't self-correct under truncation alone). 74–79% of pairs self-correct under truncation alone at `pre_setup`/`setup`, before any

patching — a finding in its own right, separate from the patching result. The within-problem vs. cross-problem gap at the cumulative stages (50% vs. 0%) is large and directionally clean. The per-layer scan shows a broad plateau (25–42%) across layers 0–32 with a falloff to 0–8% from layer 36 onward — not the localized mid-layer band that was predicted, but a real and differentiated shape.

Two open methodological questions — both unresolved, both cheap to resolve

1. Baseline reproducibility. The `already_flipped` count for `computation_only` and the cumulative stages differs between Block A and Block B on the identical 19 pairs and the identical no-patch baseline check (6 vs. 4; 5 vs. 2; 5 vs. 2). This affects the denominators underlying every flip-rate computed at those stages, including the cross-problem gap currently described as the project's cleanest result.

2. The Block D floor. The control designed to establish a near-zero "no real effect" baseline for single-layer patches instead shows a 20–60% range at n=5 — overlapping substantially with the 0–42% range of the main per-layer signal. Until this floor is established at comparable n to the main scan, no single layer in the State-4 curve can be confidently separated from floor noise.

Both of these are most directly addressed by the same experiment: a self-patch identity check on the correct-group pairs (inject a trace's own activations into itself; confirm the output doesn't move). If it doesn't move, the floor in Block D and the reproducibility gap in States A/B both point toward genuine generation-level nondeterminism (sampling settings, fp16 drift) rather than a mechanism bug — still worth fixing, but a different and more tractable fix than a contaminated patching mechanism. If the self-patch output *does* move, the patching mechanism itself is introducing noise, and that noise is present in every number in this document.

The regex fix — proposed, not yet tested

The `\boxed{}` classification fix (Section 3) has a specific, checkable failure mode: the four-backslash pattern as proposed matches nothing, leaving the original misclassification in place while appearing fixed. A ten-second test against a known `\boxed{}`-containing pair, checking both the two-backslash and four-backslash candidates against the existing `PATTERNS` dict's convention for other LaTeX commands, resolves this before it affects anything else.

The immediate next step

Per the most recent execution plan: a focused per-layer scan at layers 33 through 37 on the existing 19 pairs, at `computation_only`. This is small — at the corrected timing constant (22, not 8), this is closer to 45–60 minutes than the originally-estimated 12, and that revised estimate should be checked against the actual wall-clock time of this run before any larger run is scheduled. Its purpose is to locate, at finer granularity than the 4-layer spacing of the

State-4 scan, exactly where the transition from the 25–42% plateau to the 0–8% falloff occurs. That specific layer — wherever it lands between 32 and 36 — is the most concrete mechanistic detail this project has produced to date.

This document's Section 10 will be incomplete the moment that scan produces a curve. That is by design: this is a record of where the project has been, not a forecast of where it's going. The next entry in this document is a number between 32 and 36, and it does not exist yet.

Appendix — Bug Quick Reference

#	Bug	Location	Fix
1	`dtype` instead of `torch_dtype`	model load	`torch_dtype=torch.f`
2	`[VERIFY]` `bnb_4bit_compute_torch_dtype` typo	BitsAndBytesConfig	`bnb_4bit_compute_dt`
3	`re.split('\n+', ...)` segmentation drift	`get_stage_masks`	character-exact line wal
4	setup truncation = `max(setup_idx)+1`	`build_cumulative_masks`	first non-setup token inc
5	cumulative masks not clipped to `c_trace_len`	`build_cumulative_masks`	clip before truncation ca
6	shuffled positions not clipped	shuffled control	`min(trunc_len - p_l`
7	dual normalizers (`_clean_num` vs `normalize_answer`)	answer comparison	removed `_clean_num`
8	`p_len` recomputed in hook (`run_per_layer` only)	per-layer hook	use precomputed values
9	`\boxed{}` nested-brace parsing	`extract_gt`	depth-counted brace ma

#	Bug	Location	Fix
10	`max_new_tokens=150`	all 4 generate calls	`MAX_NEW_TOKENS=2048`
11	`\boxed{}` segmentation regex backslash count	`PATTERNS["conclusion"]`	2 vs 4 backslashes, untes
12	cross-problem adjacent-pair selection	`run_experiment` cross-problem	`(pi + len(pairs)//2`
13	top-up collection, no dedup	top-up block	filter against existing qu
14	`ds_hard` undefined on top-up path	top-up block	explicit reload guard
15	regex test on `pairs[0]` may false-negative	diagnostic cell	search all pairs for `boxe`
16	segmentation check passes silently on zero output	validation cell	assert at least one `\boxe`
17	`greedy_budget=300` too low for top-up	top-up collection	raised to 500
18	per-layer timing constant (8s) wrong by 3-5x	`run_per_layer` estimate	constant changed to 22
19	inverted control flip metric	`run_experiment` flip logic	target-aware: `not answ` control
20	A/B `already_flipped` reproducibility gap	baseline check, `computation_only`+cumulative	**none yet — likely `do_s` nondeterminism**
21	Block D floor (20-60%) overlaps Block C signal (0-42%)	per-layer correct-group control	**none yet — needs same**